# Análisis de la complejidad y eficiencia de algoritmos

## Análisis y Diseño de Datos y Algoritmos
## Estructuras de Datos y Algoritmos

ESCUELA TÉCNICA SUPERIOR DE
INGENIERÍA INFORMÁTICA
Departamento de Lenguajes y Sistemas Informáticos
Curso 2026-2027

# Contenido

- Introducción
- Análisis de la complejidad
- Complejidad de sumatorios
- Complejidad de recurrencias
  - Relación sumatorios y recurrencias
  - Ejemplos
- Caso mejor, peor y medio

# Introducción

# Introducción

¿Qué algoritmo escoger?

- Que sea fácil de entender, codificar, depurar, verificar y mantener
- Que permita usar eficientemente los recursos del ordenador y se ejecute en el menor tiempo posible

(normalmente, objetivos contrapuestos)

Por ello, necesitamos analizar el tiempo de ejecución de los algoritmos

# Introducción

- Análisis de la complejidad de un algoritmo: estudio del **tiempo que tarda** en ejecutarse en función del **tamaño del problema** que se está resolviendo

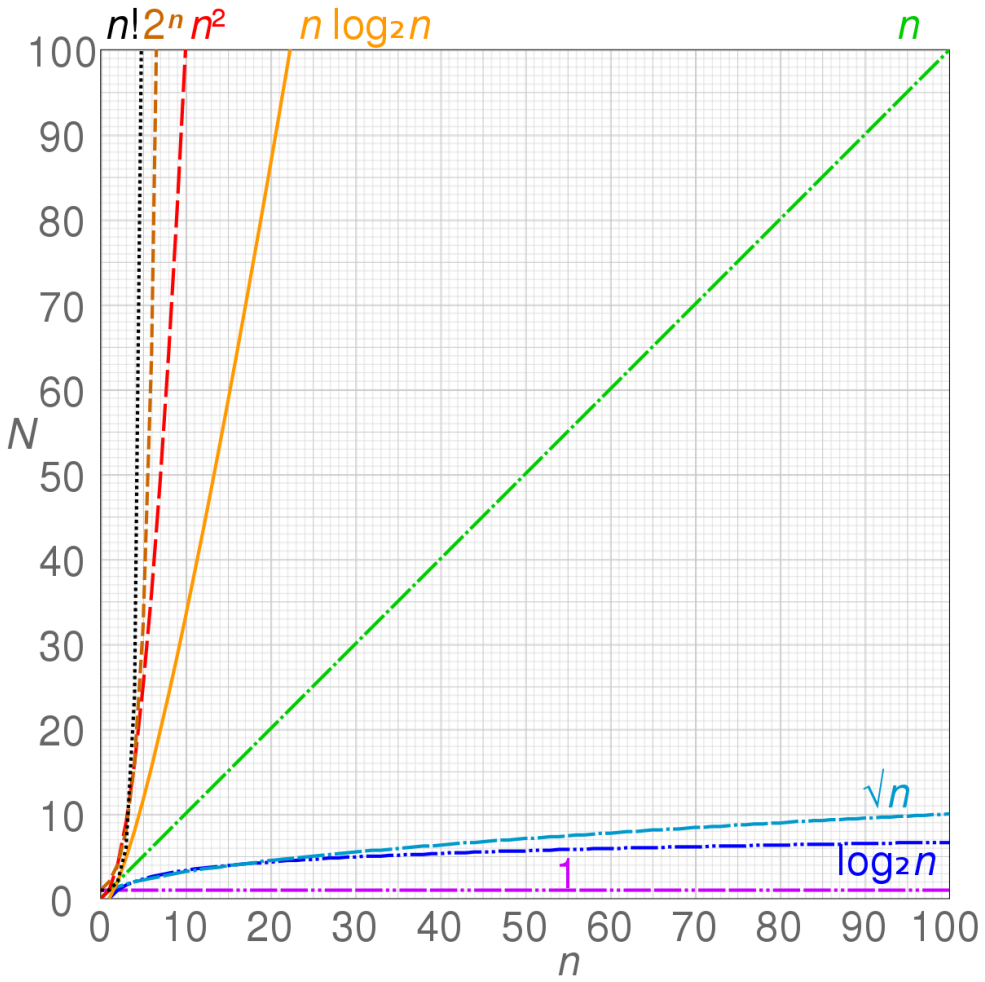
- Se representa la idea mediante la función *T(n)*, monótona creciente, siendo *n* el tamaño del problema

- ¿Cuál es el comportamiento de *T(n)* para valores grandes de *n*? → ¿en qué clase de equivalencia está?

# Introducción

Jerarquía de órdenes de complejidad exactos

- O(1) constante
- O(log n) logarítmico
- O(n) lineal
- O(n log n) cuasi-lineal
- O($n^2$) cuadrático
- O($n^a$) polinómico (a > 2)
- O($a^n$) exponencial (a > 1)
- O(n!) factorial
- O($n^n$)



# Introducción

- Efecto de duplicar el tamaño del problema

| T(n) | n=100 | n=200 |
|---|---|---|
| $k_1 \log n$ | 1 s | 1,15 s |
| $k_2 n$ | 1 s | 2 s |
| $k_3 n \log n$ | 1 s | 2,30 s |
| $k_4 n^2$ | 1 s | 4 s |
| $k_5 n^3$ | 1 s | 8 s |
| $k_6 2^n$ | 1 s | $1{,}27 \cdot 10^{30}$ s |

- Efecto de duplicar el tiempo disponible

| T(n) | t = 1 s | t = 2 s |
|---|---|---|
| $k_1 \log n$ | n=100 | n=10000 |
| $k_2 n$ | n=100 | n=200 |
| $k_3 n \log n$ | n=100 | n=178 |
| $k_4 n^2$ | n=100 | n=141 |
| $k_5 n^3$ | n=100 | n=126 |
| $k_6 2^n$ | n=100 | n=101 |

# Análisis de la complejidad

# Análisis de la complejidad

Tamaño *n* de un problema *p*:

- Medida de la **cantidad de información** necesaria para representarlo
- Definido a partir de las propiedades del problema
- Cada sub-problema o siguiente estado es menor que el problema o estado original

```
Boolean cm(List<Integer> lis, Integer a) {
    Boolean r = false;
    for(Integer e: lis){
        r = e%a==0;
        if(r) break;
    }
    return r;
}
```

# Análisis de la complejidad

- Secuencia de bloques:

```
s1;
s2;
…
sk;
```

$$T_s(n) = T_{s1}(n) + T_{s2}(n) + \cdots + T_{sk}(n)$$

- Bloque *while*:

```
while (g) {
   s;
}
```

$$T_w(n) = T_g + \sum_{i \in 1} (T_g + T_s(i))$$

(siendo I el conjunto de valores que va tomando el tamaño en las sucesivas iteraciones)

- Bloque *if*:

```
if (g) {
   s1;
} else {
   s2;
}
```

$$T_{if}^{p} = T_g^{p} + max(T_{s1}^{p}, T_{s2}^{p})$$

$$T_{if}^{m} = T_g^{m} + min(T_{s1}^{m}, T_{s2}^{m})$$

$$T_{if}^{md} = T_g^{md} + f_1 T_{s1}^{md} + f_2 T_{s2}^{md}$$

(siendo $f_i$ la frecuencia de ejecución del bloque *si* ($i$=1,2))

# Análisis de la complejidad

- Complejidad de un algoritmo recursivo **sin** memoria

  Ecuación de recurrencias { complejidad de las llamadas recursivas / complejidad del cuerpo del algoritmo

$$T(n) = T\big(t_0(n)\big) + T\big(t_1(n)\big) + \cdots + T\big(t_{k-1}(n)\big) + f(n)$$

- Complejidad de un algoritmo recursivo **con** memoria

  Sumatorio de la complejidad de todos los subproblemas necesarios para resolver el problema dado

$$\Theta\Big(\sum_{p \epsilon \mathcal{P}} f(n_p)\Big)$$

# Complejidad de sumatorios

# Complejidad de sumatorios

Para calcular la complejidad de sumatorios (bucles) usaremos las siguientes propiedades dependiendo de si la variable recorre una progresión aritmética o geométrica

$$\sum_{x \in pa(a,r)}^{n} x^d \log^p x =_\infty \frac{1}{r(d+1)} n^{d+1} \log^p n$$

$$\sum_{x \in pg(a,r)}^{n} x^d \log^p x \cong_\infty \begin{cases} \log^{p+1} n, & \text{si } d = 0, r > 1 \\ n^d \log^p n, & \text{si } d > 0, r > 1 \end{cases}$$

*pa(a,r)* es progresión aritmética

(*x* recorre la secuencia $a+ri$, $i=0,1,2\ldots$ hasta $x=n$)

*pg(a,r)* es progresión geométrica

(*x* recorre la secuencia $ar^i$, $i=0,1,2\ldots$ hasta $x=n$)

# Complejidad de sumatorios

Para calcular la complejidad de sumatorios (bucles) usaremos las siguientes propiedades dependiendo de si la variable recorre una progresión aritmética o geométrica

$$\sum_{x \in pa(a,r)}^{n} x^d \log^p x =_\infty \frac{1}{r(d+1)} n^{d+1} \log^p$$

$$\sum_{x \in pg(a,r)}^{n} x^d \log^p x \cong_\infty \begin{cases} \log^{p+1} n, & \text{si } d = 0, \\ n^d \log^p n, & \text{si } d > 0, \end{cases}$$

```
i= 1;
while(i <= n) {
    s = s + i;
    i = i + r;
}
```

```
i = n;
while(i >= 1) {
    s = s + i;
    i = i / r;
}
```

*pa(a,r)* es progresión aritmética

(*x* recorre la secuencia $a+ri$, $i=0,1,2\ldots$ hasta $x=n$)

*pg(a,r)* es progresión geométrica

(*x* recorre la secuencia $ar^i$, $i=0,1,2\ldots$ hasta $x=n$)

# Complejidad de recurrencias

# Complejidad de recurrencias

Consideramos dos tipos de recurrencias, dependiendo del tamaño del subproblema con respecto al problema (*n-b* o *n/b*)

## Recurrencias lineales

$$T(n) = aT(n-b) + n^d \log^p n$$

$$\Theta(T(n)) = \begin{cases} a^{n/b} \log^p n, & \text{si } a > 1 \\ n^{d+1} \log^p n, & \text{si } a = 1 \\ n^d \log^p n, & \text{si } a < 1 \end{cases}$$

## Recurrencias no lineales

$$T(n) = aT(n/b) + n^d \log^p n$$

$$\Theta(T(n)) = \begin{cases} n^{\log_b a}, & \text{si } a > b^d \\ n^d \log^{p+1} n, & \text{si } a = b^d \\ n^d \log^p n, & \text{si } a < b^d \end{cases}$$

# Complejidad de recurrencias

Consideramos dos tipos de recurrencias, dependiendo del tamaño del subproblema con respecto al problema ($n$-$b$ o $n/b$)

## Recurrencias lineales

$$T(n) = aT(n-b) + n^d \log^p n$$

$$\Theta(T(n)) = \begin{cases} a^{n/b} \log^p n, & \text{si } a > 1 \\ n^{d+1} \log^p n, & \text{si } a = 1 \\ n^d \log^p n, & \text{si } a < 1 \end{cases}$$

```
public static Long factorial(Integer n) {
    if (n==0) {
        return 1L;
    }else {
        return n*factorial(n-1);
    }
}
```

# Complejidad de recurrencias

Consideramos dos tipos de recurrencias, dependiendo del tamaño del subproblema con respecto al problema ($n$-$b$ o $n/b$)

## Recurrencias no lineales

$$T(n) = aT(n/b) + n^d \log^p n$$

$$\Theta(T(n)) = \begin{cases} n^{\log} \\ n^d \, l \\ n^d \, l \end{cases}$$

```
public static Integer busquedaBinariaRec(List<Integer> lista,
      Integer objetivo, Integer i, Integer j) {
   if (i < j) {
      Integer k = (i+j)/2;
      if (lista.get(k).equals(objetivo)){
         return k;
      }else if (lista.get(k) < objetivo) {
         return busquedaBinariaRec(lista, objetivo,k+1,j);
      }else if (lista.get(k) > objetivo) {
         return busquedaBinariaRec(lista, objetivo,i,k);
      }
   }
   return -1;
}
```

# Complejidad de recurrencias

¿Y si tenemos llamadas recursivas de distinto tamaño?

```
public static Integer fibonacciR(Integer n) {
    Integer r = null;
    if (n <= 1) {
        r = n;
    }else {
        r = fibonacciR(n - 1) + fibonacciR(n - 2);
    }
    return r;
}
```

# Complejidad de recurrencias

Recurrencias lineales (aproximaciones):

$$T(n) = a_1 T(n - b_1) + a_2 T(n - b_2) + \ldots + a - kT(n - b_k) + g(n), b_i < b_{i+1}$$

Verifica:

$$\Theta(R(n)) < \Theta(T(n)) < \Theta(S(n))$$

Donde:

$$R(n) = (a_1 + a_2 + \ldots + a_k) R(n - b_k) + g(n)$$

$$S(n) = (a_1 + a_2 + \ldots + a_k) S(n - b_1) + g(n)$$

# Complejidad de recurrencias

Recurrencias no lineales (aproximaciones):

$$T(n) = a_1 T(\frac{n}{b_1}) + T(\frac{n}{b_2}) + \ldots + T(\frac{n}{b_k}) + g(n), b_i < b_{i+1}$$

Verifica:

$$\Theta(R(n)) < \Theta(T(n)) < \Theta(S(n))$$

Donde:

$$R(n) = (a_1 + a_2 + \ldots + a_k) R(\frac{n}{b_k}) + g(n)$$

$$S(n) = (a_1 + a_2 + \ldots + a_k) S(\frac{n}{b_1}) + g(n)$$

# Relación sumatorios y recurrencias

- Debido a la transformación de recursivo final a iterativo los órdenes de complejidad de los sumatorios son iguales a los de sus respectivas recurrencias

- Pueden considerarse las siguientes aproximaciones:

$$T(n) = T(n-b) + n^d \log^p n \equiv \sum_{x \in pa(a,r)}^{n} x^d \log^p x$$

$$T(n) = T(n/b) + n^d \log^p n \equiv \sum_{x \in pg(a,r)}^{n} x^d \log^p x$$

# Complejidad de recurrencias

Ejemplo:

```
public static Double f(Integer n, Double a) {
   Double r;
   if (n == 1) {
      r = a;
   }else {
      r = f(n/2,a+1) - f(n/2,a-1);
      for(Integer i=1;i<=n;i++) {
         r+= a*i;
      }
   }
   return r;
}
```

Complejidad del cuerpo del algoritmo (descartando llamadas recursivas):

1. Cuerpo del bucle → bloque básico

$$\sum_{i=1}^{n} 1 = n \in \Theta(n)$$

2. Exterior del bucle → bloque básico

Cuerpo del algoritmo: $\Theta(1+n) = \Theta(n)$

A partir de aquí, ecuación de recurrencia, y aplicación de fórmulas anteriores →

$$T(n) = 2T(n/2) + n$$

$$\Theta(T(n)) = \Theta(n \log n)$$

# Complejidad de recurrencias

Ejemplo: complejidad del algoritmo para calcular los números de Fibonacci

$$f(n) = \begin{cases} n, & \text{si } n \leq 1 \\ f(n-1) + f(n-2), & \text{si } n > 1 \end{cases}$$

```
public static Integer fibonacciR(Integer n) {
    Integer r = null;
    if (n <= 1) {
        r = n;
    }else {
        r = fibonacciR(n - 1) + fibonacciR(n - 2);
    }
    return r;
}
```

# Complejidad de recurrencias

Ejemplo: complejidad del algoritmo para calcular los números de Fibonacci

$$f(n) = \begin{cases} n, & \text{si } n \leq 1 \\ f(n-1) + f(n-2), & \text{si } n > 1 \end{cases}$$

Recurrencia correspondiente: $T(n) = T(n-1) + T(n-2) + 1$

Puede ser acotada por las recurrencias:

$$T(n) = 2T(n-2) + 1, T(n) = 2T(n-1) + 1$$

Luego: $\Theta(2^{n/2}) < \Theta(T(n)) < \Theta(2^n)$

Con memoria sería: $\Theta(\sum_{1}^{n} 1) = \Theta(n)$

# Complejidad de recurrencias

Ejemplo:

Asumiendo, para todo $i$ $\Theta(A(i) = \Theta(i)$ y $\Theta(B(i,n)) = \Theta(n)$

```
public static Integer F(Integer n) {
   Integer x, j, i;
   if (n < 10) {
      i = n;
   }else {
      i = 1;
      j = 0;
      while ((i * i) <= n) {
         j = j + A(i);
         i = i + 1;
      }
      x = n;
      while (x > 1) {
         j = j + x;
         x = x / 4;
         for (Integer l=1; l<=n;l++) {
            j = j * B(l,n);
         }

      }
      i = 2 * F(n/2) + j; //i = F(n/2) + F(n/2) + j;
   }
   return i;
}
```

Primer bucle while

$$\sum_{i=1}^{\sqrt{n}} i \cong_\infty (\sqrt{n})^2 \in \Theta(n)$$

Bucles anidados

$$\sum_{x \in pg(1,4)} \sum_{i=1}^{n} n \cong_\infty n \sum_{x \in pg(1,4)} \sum_{i=1}^{n} 1 \cong_\infty n^2 \sum_{x \in pg(1,4)} 1 \in \Theta(n^2 \log n)$$

Algoritmo completo, en dos versiones, sin memoria

$$T(n) = T(n/2) + n^2 \log n, T(n) \in \Theta(n^2 \log n)$$

$$T(n) = 2T(n/2) + n^2 \log n, T(n) \in \Theta(n^2 \log n)$$

# Complejidad de recurrencias

Ejemplo:

Asumiendo, para todo $i$ $\Theta(A(i) = \Theta(i)$ y $\Theta(B(i,n)) = \Theta(n)$

```
public static Integer F(Integer n) {
   Integer x, j, i;
   if (n < 10) {
      i = n;
   }else {
      i = 1;
      j = 0;
      while ((i * i) <= n) {
         j = j + A(i);
         i = i + 1;
      }
      x = n;
      while (x > 1) {
         j = j + x;
         x = x / 4;
         for (Integer l=1; l<=n;l++) {
            j = j * B(l,n);
         }

      }
      i = 2 * F(n/2) + j; //i = F(n/2) + F(n/2) + j;
   }
   return i;
}
```

Primer bucle while

$$\sum_{i=1}^{\sqrt{n}} i \cong_\infty (\sqrt{n})^2 \in \Theta(n)$$

Bucles anidados

$$\sum_{x \in pg(1,4)}^{n} \sum_{i=1}^{n} n \cong_\infty n \sum_{x \in pg(1,4)}^{n} \sum_{i=1}^{n} 1 \cong_\infty n^2 \sum_{x \in pg(1,4)}^{n} 1 \in \Theta(n^2 \log n)$$

Con memoria, para ambas versiones:

$$\sum_{x \in pg(1,2)}^{n} x^2 \log x \in \Theta(n^2 \log n)$$

# Caso mejor, peor y medio

# Caso mejor, peor y medio

Problemas de mismo tamaño pueden tardar tiempos diferentes

Diferentes casos de ejecución:

- Peor → $T_p(n)$
- Mejor → $T_m(n)$
- Medio → $T_d(n)$

El caso medio depende de la distribución de probabilidad de los problemas, $f(p)$:

$$T_d(n) = \sum_{p \in P_n} T(n_p) f(p)$$

Ejemplo

```
Boolean contieneMultiplo(List<Integer> ls, Integer a) {
   Boolean r = false;
   for(Integer e:ls) {
      if(e%a==0) {
         r = true;
         break;
      }
   }
   return r;
}
```

Caso mejor: primer elemento múltiplo de a → complejidad $\Theta(1)$

Caso peor: no hay múltiplo de a → complejidad $\Theta(n)$

Caso medio: $\Theta(1)$

# Caso mejor, peor y medio

Problemas de mismo tamaño pueden tardar tiempos diferentes

Diferentes casos de ejecución:

- Peor → $T_p(n)$
- Mejor → $T_m(n)$
- Medio → $T_d(n)$

El caso medio depende de la distribución de probabilidad de los problemas, $f(p)$:

$$T_d(n) = \sum_{p \in P_n} T(n_p) f(p)$$

Ejemplo SIN OPTIMIZAR

```
Boolean contieneMultiploSO(List<Integer> ls, Integer a) {
   Boolean r = false;
   for(Integer e:ls) {
      r = r || e%a==0;
   }
   return r;
}
```

Caso mejor: complejidad $\Theta(n)$

Caso peor: complejidad $\Theta(n)$

Caso medio: complejidad $\Theta(n)$

# Análisis de la complejidad y eficiencia de algoritmos

## Análisis y Diseño de Datos y Algoritmos
## Estructuras de Datos y Algoritmos

ESCUELA TÉCNICA SUPERIOR DE
INGENIERÍA INFORMÁTICA
Departamento de Lenguajes y Sistemas Informáticos
Curso 2026-2027